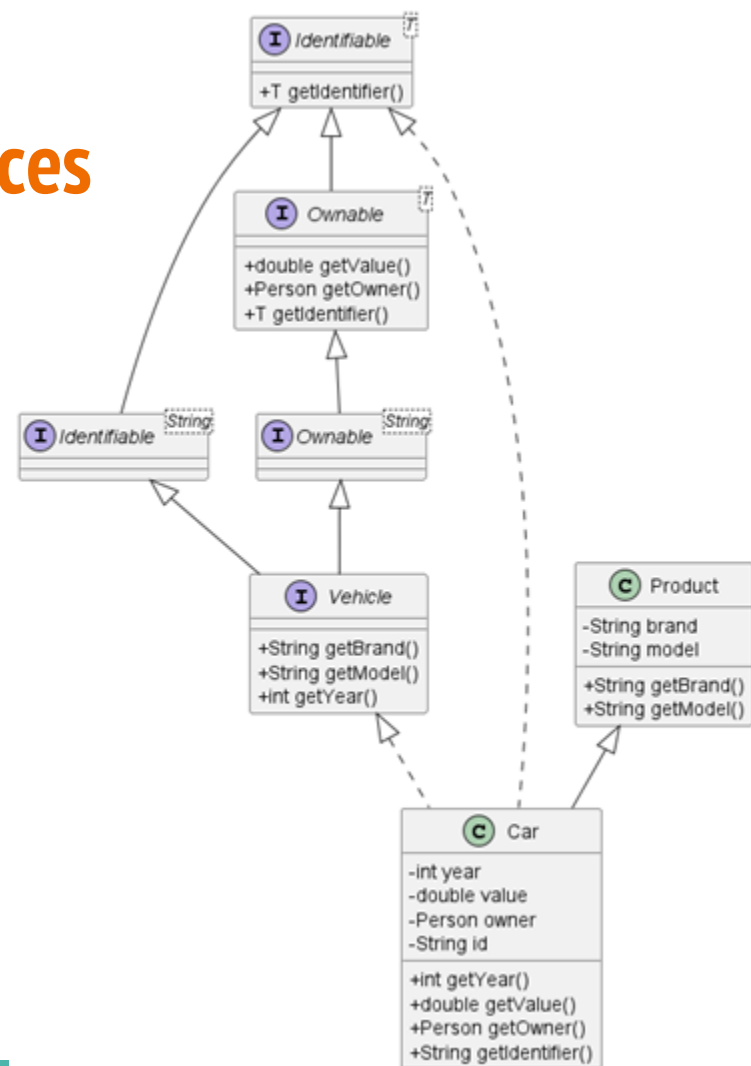

Exceptions and Assertions

Lukas Holik

lukasholik@cs.aau.dk

(based on slides of Gabriela Montoya, thanks)

Classes, Inheritance, Interfaces



Exception

- Abnormal condition that arises when the normal path of execution is not defined
 - An **exception** is **thrown**
 - That condition needs to be **handled (caught)**
- Represented using the class **java.lang.Exception**

Example

```
1. import java.util.ArrayList;

2. class Course {
3.     private String code;
4.     private String title;
5.     private ArrayList<String> goals;

6.     public Course (String c, String t) {
7.         this.code = c;
8.         this.title = t;
9.         this.goals = new ArrayList<String>();
10.    }
11.    public void addGoal(String g) {
12.        goals.add(g);
13.    }
14.    public void removeLastGoal() {
15.        goals.remove(goals.size()-1);
16.    }
17. }
```

```
20. public static void main(String[] args) {
21.     Course c = new Course("OOP",
22.         "Object-oriented programming");

23.     c.removeLastGoal();
24.     System.out.println("Done");
25. }
26. }
```

```
>java -cp class Course
Exception in thread "main" java.lang.IndexOutOfBoundsException: Index -1 out of bounds for length 0
    at java.base/jdk.internal.util.Preconditions.outOfBounds(Preconditions.java:64)
    at java.base/jdk.internal.util.Preconditions.outOfBoundsCheckIndex(Preconditions.java:70)
    at java.base/jdk.internal.util.Preconditions.checkIndex(Preconditions.java:266)
    at java.base/java.util.Objects.checkIndex(Objects.java:359)
    at java.base/java.util.ArrayList.remove(ArrayList.java:504)
    at Course.removeLastGoal(Course.java:17)
    at Course.main(Course.java:24)
```

Uncaught exception!

java.util.ArrayList

```
>java -cp class Course
Exception in thread "main" java.lang.IndexOutOfBoundsException: Index -1 out of bounds for length 0
    at java.base/jdk.internal.util.Preconditions.outOfBounds(Preconditions.java:64)
    at java.base/jdk.internal.util.Preconditions.outOfBoundsCheckIndex(Preconditions.java:70)
    at java.base/jdk.internal.util.Preconditions.checkIndex(Preconditions.java:266)
    at java.base/java.util.Objects.checkIndex(Objects.java:359)
    at java.base/java.util.ArrayList.remove(ArrayList.java:504)
    at Course.removeLastGoal(Course.java:17)
    at Course.main(Course.java:24)
```

remove

```
public E remove(int index)
```

Removes the element at the specified position in this list. Shifts any subsequent elements to the left (subtracts one from their indices).

Specified by:

remove in interface List<E>

Overrides:

remove in class AbstractList<E>

Parameters:

index - the index of the element to be removed

Returns:

the element that was removed from the list

Throws:

IndexOutOfBoundsException - if the index is out of range (`index < 0` || `index >= size()`)

```
21.    Course c = new Course("OOP",
22.        "Object-oriented programming");
23.    c.removeLastGoal();
```

main

```
16.    public void removeLastGoal() {
17.        goals.remove(goals.size()-1);
18.    }
```

Check before performing an action...

```
16.    public void removeLastGoal() {  
17.        int size = goals.size();  
18.        if (size > 0) {  
19.            goals.remove(size-1);  
20.        }  
21.    }
```

- It avoids the uncaught exception
- But...
 - what is done by the method when there are not goals?
 - Shouldn't the caller be informed of the situation?

Check before performing an action...

```
16.    public void removeLastGoal() {  
17.        int size = goals.size();  
18.        if (size > 0) {  
19.            goals.remove(size-1);  
20.        } else {  
21.            System.out.println("You can't remove goals from a course without any goals");  
22.        }  
23.    }
```

- It avoids the uncaught exception
- But...
 - Is the caller informed?

Handle the abnormal condition

try-block: code that performs the action that can cause an exception
catch-block: code that handles the exception

```
16.    public void removeLastGoal() {  
17.        int size = goals.size();  
18.        try {  
19.            goals.remove(size-1);  
20.        } catch (IndexOutOfBoundsException e) {  
21.            System.out.println("You can't remove goals from a course without any goals");  
22.        }  
23.    }
```

- It handles the exception
- But...
 - Can the possible issue be rectified and continue with the program?

Let the caller handle the abnormal condition

A **throws** clause is added to the method declaration

```
16.    public void removeLastGoal() throws IndexOutOfBoundsException {  
17.        int size = goals.size();  
18.        goals.remove(size-1);  
19.    }
```

- It handles the exception, letting the caller handle it
- The method declaration lets the caller know that an exception could be thrown
 - Does the caller has all the information needed?
 - Does this exception type make sense for the caller?

A caller can be better prepared to handle the abnormal condition

```
16.    public void removeLastGoal() throws Exception {
17.        int size = goals.size();
18.        try {
19.            goals.remove(size-1);
20.        } catch (IndexOutOfBoundsException e) {
21.            throw new Exception("Attempted to remove "
22.                + "goals from a course without goals");
23.        }
24.    }
```

The **throw** statement throws an exception instance
The object has relevant information that is passed to the caller

```
29.        try {
30.            c.removeLastGoal();
31.        } catch (Exception e) {
32.            e.printStackTrace();
33.        }
```

```
try {
    c.removeLastGoal();
} catch (Exception e) {
    log.println(e.getMessage());
}
```

```
>java -cp class Course
java.lang.Exception: Attempted to remove goals from a course without goals
    at Course.removeLastGoal(Course.java:21)
    at Course.main(Course.java:30)
```

Attempted to remove goals from a course without goals

Using a more specific exception class

```
16.    public void removeLastGoal() {  
17.        int size = goals.size();  
18.        try {  
19.            goals.remove(size-1);  
20.        } catch (IndexOutOfBoundsException e) {  
21.            throw new CourseWithoutGoalsException("Attempted to remove "  
22.                + "goals from a course without goals");  
23.        }  
24.    }
```

removeLastGoal doesn't have a throws clause... which of the following statements is correct?

- a. We cannot add a throws clause to this method
- b. We have to add a throws clause to this method
- c. We can add a throws clause but it is not needed

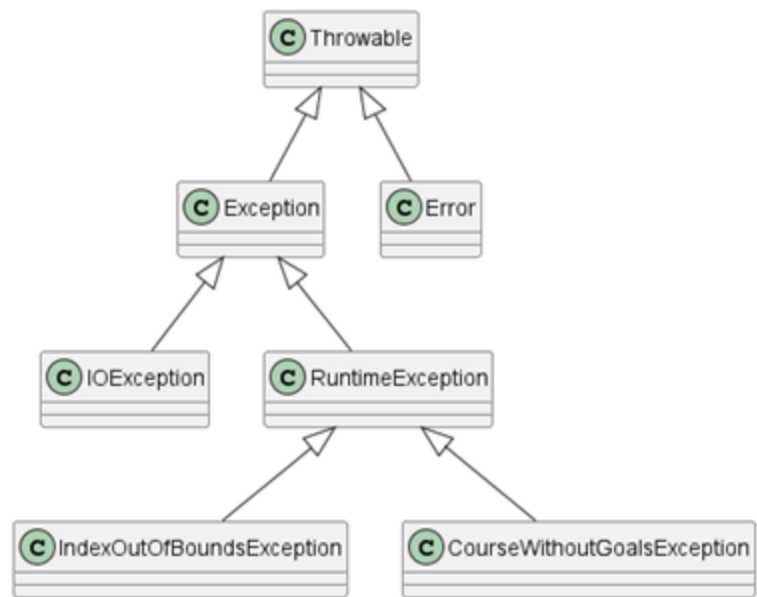
What information can the caller get from the instance of Exception?

```
try {  
    c.removeLastGoal();  
} catch (Exception e) {  
    ...  
}
```

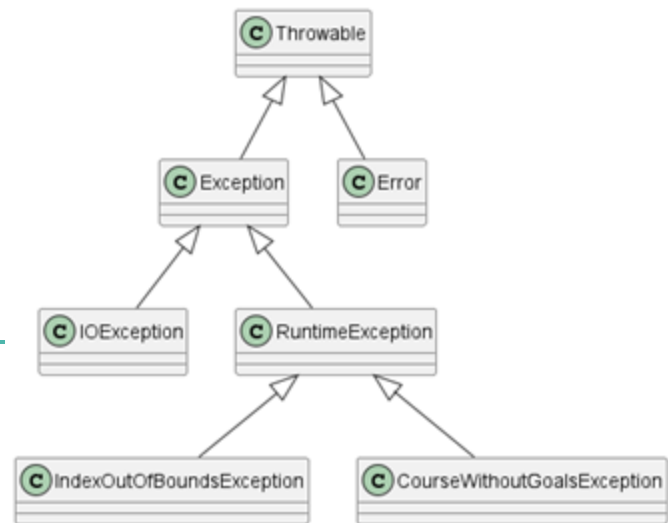
- a. A message passed to the constructor when that instance was created
- b. How to handle the exception
- c. The class name and method name where that exception arised
- d. The file and line number where that exception was thrown

Exception Hierarchy

- The top class is [java.lang.Throwable](#)
- Only instances of Throwable can be parameter of a catch block or be thrown
- **Checked** exceptions are throwables that are not RuntimeException, Error or any of their subclasses
 - These must be handled (and that is checked by the compiler)
 - E.g., IOException
- It is unlikely that the program can recover if an Error is thrown (e.g., java.lang.OutOfMemoryError).
 - Changes to the code or specifications must be done by the programmer



Exception Hierarchy

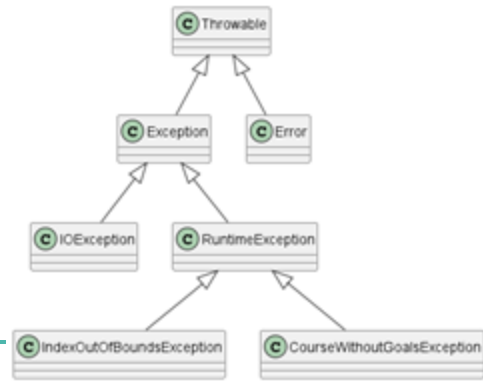


```
16. public void removeLastGoal() {  
17.     int size = goals.size();  
18.     try {  
19.         goals.remove(size-1);  
20.     } catch (IndexOutOfBoundsException e) {  
21.         throw new CourseWithoutGoalsException("Attempted to remove "  
22.             + "goals from a course without goals");  
23.     }  
24. }
```

```
public static void main(String[] args) {  
    Course c = new Course("OOP",  
        "Object-oriented programming");  
    ...  
    c.removeLastGoal();  
}
```

main does not have to handle the **CourseWithoutGoalsException** that could be thrown by the **removeLastGoal** method

Exception Hierarchy



```
16. public void removeLastGoal() throws CourseWithoutGoalsException{
17.     int size = goals.size();
18.     try {
19.         goals.remove(size-1);
20.     } catch (IndexOutOfBoundsException e) {
21.         throw new CourseWithoutGoalsException("Attempted to remove "
22.             + "goals from a course without goals");
23.     }
24. }
```

```
try {
    c.removeLastGoal();
} catch (Exception e) {
    e.printStackTrace();
}
```

CourseWithoutGoalsException is handled by the **first** catch block with **parameter type** CourseWithoutGoalsException or any of its supertypes in the Exception Hierarchy
Using more specific types is always preferable!

CourseWithoutGoalsException

```
class CourseWithoutGoalsException extends RuntimeException {  
    public CourseWithoutGoalsException() {  
    }  
  
    public CourseWithoutGoalsException(String msg) {  
        super(msg);  
    }  
  
    public CourseWithoutGoalsException(String msg, Throwable cause) {  
        super(msg, cause);  
    }  
  
    public CourseWithoutGoalsException(Throwable cause) {  
        super(cause);  
    }  
}
```

What methods does CourseWithoutGoalsException have?



All Methods	Instance Methods	Concrete Methods
Modifier and Type	Method	Description
final void	<code>addSuppressed(Throwable exception)</code>	Appends the specified exception to the exceptions that were suppressed in order to deliver this exception.
Throwable	<code>fillInStackTrace()</code>	Fills in the execution stack trace.
Throwable	<code>getCause()</code>	Returns the cause of this throwable or null if the cause is nonexistent or unknown.
String	<code>getLocalizedMessage()</code>	Creates a localized description of this throwable.
String	<code>getMessage()</code>	Returns the detail message string of this throwable.
StackTraceElement[]	<code>getStackTrace()</code>	Provides programmatic access to the stack trace information printed by <code>printStackTrace()</code> .
final Throwable[]	<code>getSuppressed()</code>	Returns an array containing all of the exceptions that were suppressed, typically by the <code>try-with-resources</code> statement, in order to deliver this exception.
Throwable	<code>initCause(Throwable cause)</code>	Initializes the <i>cause</i> of this throwable to the specified value.
void	<code>printStackTrace()</code>	Prints this throwable and its backtrace to the standard error stream.
void	<code>printStackTrace(PrintStream s)</code>	Prints this throwable and its backtrace to the specified print stream.
void	<code>printStackTrace(PrintWriter s)</code>	Prints this throwable and its backtrace to the specified print writer.
void	<code>setStackTrace(StackTraceElement[] stackTrace)</code>	Sets the stack trace elements that will be returned by <code>getStackTrace()</code> and printed by <code>printStackTrace()</code> and related methods.
String	<code>toString()</code>	Returns a short description of this throwable.

Which of these statements could throw an exception?

```
1. public void register(String stdId, String education) {  
2.     Student s = getStudent(stdId);  
3.     Education e = getEducation(education);  
4.     s.setEducation(e);  
5.     List<Course> cs = getInitialCourses(e);  
6.     for (Course c : cs) {  
7.         s.registerForCourse(c);  
8.     }  
9. }
```

- a. Statement in line 2
- b. Statement in line 3
- c. Statement in line 4
- d. Statement in line 5
- e. Statement in line 7

Handling exceptions

```
public void register(String stdId, String education) {  
    Student s = getStudent(stdId);  
    Education e = getEducation(education);  
    s.setEducation(e);  
    List<Course> cs = getInitialCourses(e);  
    for (Course c : cs) {  
        s.registerForCourse(c);  
    }  
}
```

```
1. try {  
2.     register(id, education1);  
3.     register(id, education2);  
4. } catch (NonExistingStudentException nese) {  
5.     System.out.println(nese.getMessage());  
6.     okayStudent=false;  
7. } catch (NonExistingEducationException neee) {  
8.     System.out.println(neee.getMessage());  
9.     okayEducation=false;  
10. } catch (AlreadyRegisteredStudentException arse) {  
11.     System.out.println(arse.getMessage());  
12.     done=true;  
13. } catch (RuntimeException re) {  
14.     System.out.println(re.getMessage());  
15.     done=false;  
16. }
```

- The **first** catch-block that handles the thrown exception is executed
- **At most one** catch-block is executed

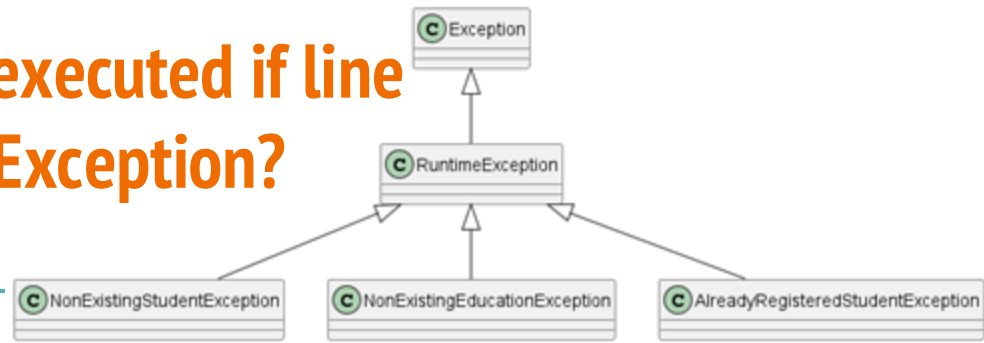
Checked exceptions should remain stable

```
1. public void register(String id, String education)
2.                                     throws RegistrationFailedException {
3.     try {
4.         handleRegistration(id, education);
5.     } catch (NonExistingStudentException nese) {
6.         throw new RegistrationFailedException(nese);
7.     } catch (NonExistingEducationException neee) {
8.         throw new RegistrationFailedException(neee);
9.     } catch (AlreadyRegisteredStudentException arse) {
10.        throw new RegistrationFailedException(arse);
11.    }
12. }
```

The implementation can evolve and include more specific exceptions, but the method register keeps one exception in its throws clause

Which of these statements are executed if line 2 throws a `NonExistingStudentException`?

```
1. try {  
2.     register(id, education1);  
3. } catch (NonExistingStudentException nese) {  
4.     System.out.println(nese.getMessage());  
5.     okayStudent=false;  
6. } catch (NonExistingEducationException neee) {  
7.     System.out.println(neee.getMessage());  
8.     okayEducation=false;  
9. } catch (RuntimeException re) {  
10.    System.out.println(re.getMessage());  
11. } catch (Exception e) {  
12.    e.printStackTrace();  
13. }  
14. System.out.println("Done");
```



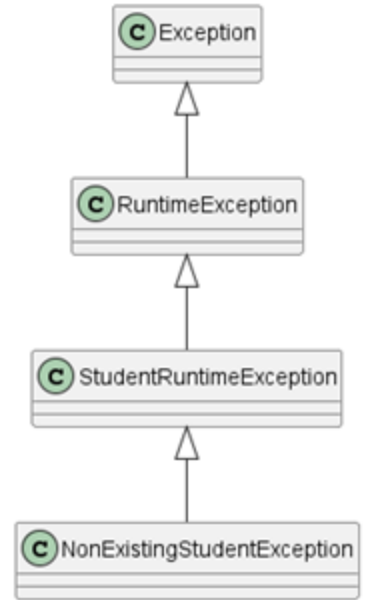
- a. Statement in line 4
- b. Statement in line 5
- c. Statement in line 7
- d. Statement in line 10
- e. Statement in line 12

Which exception type must be used as ?1

?

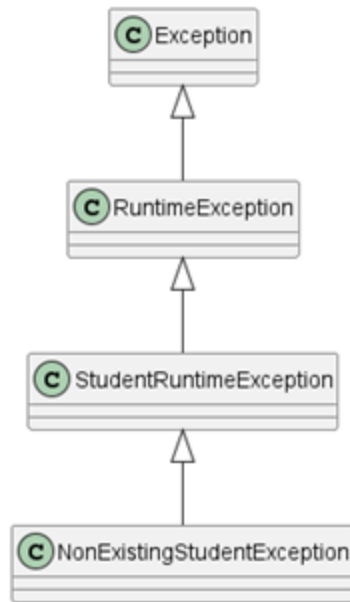
```
1. try {  
2.     register(id, education1);  
3. } catch (?1 e1) {  
4.     System.out.println(e1.getMessage());  
5. } catch (?2 e2) {  
6.     System.out.println(e2.getMessage());  
7. } catch (?3 e3) {  
8.     System.out.println(e3.getMessage());  
9. } catch (?4 e4) {  
10.    System.out.println(e4.getMessage());  
11. }
```

- a. Exception
- b. RuntimeException
- c. StudentRuntimeException
- d. NonExistingStudentException



In which order must the exceptions in this hierarchy be caught?

```
1. try {  
2.     register(id, education1);  
3. } catch (?1 e1) {  
4.     System.out.println(e1.getMessage());  
5. } catch (?2 e2) {  
6.     System.out.println(e2.getMessage());  
7. } catch (?3 e3) {  
8.     System.out.println(e3.getMessage());  
9. } catch (?4 e4) {  
10.    System.out.println(e4.getMessage());  
11. }
```



From the **most specific** to the most general


```
public FileReader(String fileName)
    throws FileNotFoundException
```

Load courses from a file

loadCourses must handle the
FileNotFoundException that could be thrown by the
FileReader constructor

```
class Course {
    private static ArrayList<Course> courses = new ArrayList<Course>();
    ...
    public static void addCourse(Course c) {
        courses.add(c);
    }
    public static void loadCourses(String fileName) throws FileNotFoundException {
        FileReader fr = new FileReader(fileName);
        Scanner s = new Scanner(fr);
        while(s.hasNextLine()) {
            String line = s.nextLine();
            StringTokenizer st = new StringTokenizer(line, ",");
            String code = st.nextToken();
            String title = st.nextToken();
            Course c = new Course(code, title);
            addCourse(c);
        }
    }
}
```

OOP, Object-oriented programming
SU, Systems Development
DEB, Design and evaluation of user interfaces

FileNotFoundException is a **checked** exception (and a subclass of IOException)

main must handle the instance of **FileNotFoundException** that could be thrown by the loadCourses method

```
public static void loadCourses(String fileName) throws FileNotFoundException {  
    FileReader fr = new FileReader(fileName);  
    Scanner s = new Scanner(fr);  
    while(s.hasNextLine()) {  
        String line = s.nextLine();  
        StringTokenizer st = new StringTokenizer(line, ",");  
        String code = st.nextToken();  
        String title = st.nextToken();  
        Course c = new Course(code, title);  
        addCourse(c);  
    }  
}
```

```
public static void main(String[] args) {  
    boolean done = false;  
    do {  
        System.out.println("File name: ");  
        String fileName = (new Scanner(System.in)).nextLine();  
        try {  
            loadCourses(fileName);  
            showCourses();  
            done = true;  
        } catch (FileNotFoundException fnfe) {  
            System.out.println("Course file not found, please try  
again!");  
        }  
    } while (!done);  
}
```

There is one file with information about

courses: **courses1.txt**.

Is the user given the opportunity to correct the file name if the user provides a wrong name (e.g., courses.txt)?

- a. No
- b. Yes

```
public static void loadCourses(String fileName) throws FileNotFoundException {  
    FileReader fr = new FileReader(fileName);  
    Scanner s = new Scanner(fr);  
    while(s.hasNextLine()) {  
        String line = s.nextLine();  
        StringTokenizer st = new StringTokenizer(line, ",");  
        String code = st.nextToken();  
        String title = st.nextToken();  
        Course c = new Course(code, title);  
        addCourse(c);  
    }  
}
```

```
public static void main(String[] args) {  
    boolean done = false;  
    do {  
        System.out.println("File name: ");  
        String fileName = (new Scanner(System.in)).nextLine();  
        try {  
            loadCourses(fileName);  
            showCourses();  
            done = true;  
        } catch (FileNotFoundException fnfe) {  
            System.out.println("Course file not found, please try  
again!");  
        }  
    } while (!done);  
}
```

Please provide your answers using [this form](#)

How does the execution of the class with the following main finish when the file with the courses is empty?

- a. Normally
- b. With an exception or error

```
public static void loadCourses(String fileName) throws FileNotFoundException {  
    FileReader fr = new FileReader(fileName);  
    Scanner s = new Scanner(fr);  
    while(s.hasNextLine()) {  
        String line = s.nextLine();  
        StringTokenizer st = new StringTokenizer(line, ",");  
        String code = st.nextToken();  
        String title = st.nextToken();  
        Course c = new Course(code, title);  
        addCourse(c);  
    }  
}
```

```
public static void main(String[] args) {  
    boolean done = false;  
    do {  
        System.out.println("File name: ");  
        String fileName = (new Scanner(System.in)).nextLine();  
        try {  
            loadCourses(fileName);  
            showCourses();  
            done = true;  
        } catch (FileNotFoundException fnfe) {  
            System.out.println("Course file not found, please try  
again!");  
        }  
    } while (!done);  
}
```

When does the constructor throw an exception?

```
class Course {  
    private static ArrayList<Course> courses;  
    private String code;  
    private String title;  
    private ArrayList<String> goals;  
  
    public Course (String c, String t) {  
        this.code = c;  
        this.title = t;  
        this.goals = new ArrayList<String>();  
        courses.add(this);  
    }  
    public static void showCourses() {  
        for (Course c : courses) {  
            System.out.println(c);  
        }  
    }  
}
```

- a. Never
- b. Only when c is null
- c. Only when t is null
- d. When either c or t is null
- e. Always

Please provide your answers using [this form](#)

The finally block is (almost) always executed

Except when the execution ends prematurely, e.g., `System.exit(int)`

```
class Course {
    ...
    private static int savingAttempts = 0;
    public static void saveCourses(String fileName) {
        PrintWriter pw;
        try {
            pw = new PrintWriter(fileName);
            for (Course c : courses) {
                pw.println(c);
            }
            pw.flush();
            pw.close();
        } catch (FileNotFoundException fnfe) {
            throw new UnsuccessfulSavingException("Course file not found", fnfe);
        } finally {
            savingAttempts++;
        }
    }
}
```

java.lang.AutoCloseable objects in the try-with-resources

```
class Course {  
    ...  
    private static int savingAttempts = 0;  
    public static void saveCourses(String fileName) {  
  
        try (PrintWriter pw = new PrintWriter(fileName)) {  
            for (Course c : courses) {  
                pw.println(c);  
            }  
            pw.flush();  
        } catch (FileNotFoundException fnfe) {  
            throw new UnsuccessfulSavingException("Course file not found", fnfe);  
        } finally {  
            savingAttempts++;  
        }  
    }  
}
```

Class `PrintWriter`

`java.lang.Object`
`java.io.Writer`
`java.io.PrintWriter`

All Implemented Interfaces:

`Closeable`, `Flushable`, `Appendable`, `AutoCloseable`

```
class ClassCastException extends RuntimeException {  
    ...  
}
```

```
class NullPointerException extends RuntimeException  
{  
    ...  
}
```

Does the following method overrides the equals method inherited from Object?

```
public boolean equals(Object other) throws NullPointerException, ClassCastException {  
    if (other == null) {  
        throw new NullPointerException();  
    } else if (!(other instanceof Course)) {  
        throw new ClassCastException();  
    }  
    Course c2 = (Course) other;  
    return code.equals(c2.code);  
}
```

- a. No
- b. Yes


```
class EqualsCheckedException extends Exception {  
    ...  
}
```

Does the following method overrides the equals method inherited from Object?

```
public boolean equals(Object other) throws EqualsCheckedException {  
    if (other == null || !(other instanceof Course)) {  
        throw new EqualsCheckedException();  
    }  
    Course c2 = (Course) other;  
    return code.equals(c2.code);  
}
```

- a. No
- b. Yes

Assertion

The **assert** statement requires a **boolean expression** and accepts an optional error message

- A condition should hold
- If the condition does not hold, an instance of `java.lang.AssertionError` is thrown (a subclass of `java.lang.Error`)

```
class Course {  
    ...  
    public boolean equals(Object other) {  
  
        assert (other instanceof Course);  
        Course c2 = (Course) other;  
        return code.equals(c2.code);  
    }  
  
    public static void main(String[] args) {  
        Course c1 = new Course("OOP", "Object-oriented programming");  
        Circle c2 = new Circle(0, 0, 4);  
        boolean b = c1.equals(c2);  
    }  
}
```

What happens when Course is executed?

```
class Course {  
    ...  
    65.    public boolean equals(Object other) {  
  
    66.        assert (other instanceof Course) : "unexpected parameter: "+other;  
    67.        Course c2 = (Course) other;  
    68.        return code.equals(c2.code);  
    69.    }  
  
    ...  
    107.    public static void main(String[] args) {  
    108.        Course c1 = new Course("OOP", "Object-oriented programming");  
    109.        Circle c2 = new Circle(0, 0, 4);  
    110.        boolean b = c1.equals(c2);  
    111.    }  
}
```

```
>java -cp class Course  
Exception in thread "main" java.lang.ClassCastException: class Circle cannot be cast to class Course (Circle and Course  
are in unnamed module of loader 'app')  
    at Course.equals(Course.java:68)  
    at Course.main(Course.java:110)
```

Assertion checking is only done when it is enabled

```
class Course {  
    ...  
    65.    public boolean equals(Object other) {  
  
    66.        assert (other instanceof Course) : "unexpected parameter: "+other;  
    67.        Course c2 = (Course) other;  
    68.        return code.equals(c2.code);  
    69.    }  
  
    ...  
    107.    public static void main(String[] args) {  
    108.        Course c1 = new Course("OOP", "Object-oriented programming");  
    109.        Circle c2 = new Circle(0, 0, 4);  
    110.        boolean b = c1.equals(c2);  
    111.    }  
}
```

enable
assertions

Assertion errors are not handled

Indicates an error in the
implementation, what was
expected doesn't hold

```
>java -ea -cp class Course  
Exception in thread "main" java.lang.AssertionError: unexpected parameter: Circle@23fc625e  
    at Course.equals(Course.java:67)  
    at Course.main(Course.java:110)
```

The error has been corrected

```
class Course {  
    public boolean equals(Object other) {  
        if (other == null || !(other instanceof Course)) {  
            return false;  
        }  
        assert (other instanceof Course) : "unexpected parameter: "+other;  
        Course c2 = (Course) other;  
        return code.equals(c2.code);  
    }  
  
    public static void main(String[] args) {  
        Course c1 = new Course("OOP", "Object-oriented programming");  
        Circle c2 = new Circle(0, 0, 4);  
        boolean b1 = c1.equals(c2);  
        Course c3 = new Course("OOP", "Object-oriented programming");  
        boolean b2 = c1.equals(c3);  
    }  
}
```

Class invariants always hold

```
class Course {  
    private String code;  
    private String title;  
    private ArrayList<String> goals;  
  
    public Course (String c, String t) {  
        this.code = c;  
        this.title = t;  
        this.goals = new ArrayList<String>();  
        assert validCourse() : "class invariant doesn't hold: "+this.toString();  
    }  
    public void addGoal(String g) {  
        assert validCourse() : "class invariant doesn't hold: "+this.toString();  
        goals.add(g);  
        assert validCourse() : "class invariant doesn't hold: "+this.toString();  
    }  
    private boolean validCourse() {  
        return (code != null) && (title != null) && (goals != null);  
    }  
}
```

Exceptions and Assertions

- Exceptions are abnormal conditions. Exceptions are thrown and handled
- The program needs to handle checked exceptions
- Assertions are conditions that should hold. If they don't, an assertion error is thrown

```
public void removeLastGoal() {  
    int size = goals.size();  
    try {  
        goals.remove(size-1);  
    } catch (IndexOutOfBoundsException e) {  
        throw new CourseWithoutGoalsException("Attempted to remove "  
            + "goals from a course without goals");  
    }  
}
```

```
class Course {  
    ...  
    public boolean equals(Object other) {  
        if (other == null || !(other instanceof Course)) {  
            return false;  
        }  
        assert (other instanceof Course) : "unexpected parameter: "+other;  
        Course c2 = (Course) other;  
        return code.equals(c2.code);  
    }  
}
```

Appendix

Clarification about overriding methods and exceptions

- The reason why the first version of equals overrides the equals method inherited from Object, but the second one doesn't is in last of the conditions to override a method (given in slide 30 of the lecture about inheritance):

A class C **overrides** an inherited non-static method by defining its own method with the **same signature** and

- an assignment-compatible return reference type
- the same return primitive type
- the same or more relax access level
- **the same or less checked exceptions**

- The exceptions in slide 31 extend RuntimeException so they are not checked exceptions, but the exception on slide 32 extends Exception so it is checked.